LUMIÈRE COSMETIC

BÁO CÁO ĐỒ ÁN CUỐI KỲ

HỌC PHẦN: ĐIỆN TOÁN ĐÁM MÂY

Đề tài: Website quản lý mỹ phẩm mini đóng gói bằng Docker Compose

Sinh viên: [Bổ sung họ tên thành viên]

Mã sinh viên: [Bổ sung]

Lớp: [Bổ sung]

Giảng viên hướng dẫn: [Bổ sung]

Đơn vị: [Bổ sung trường / khoa]

Năm học: 2026

TÓM TẮT

Ứng dụng web quản lý cửa hàng mỹ phẩm mini với Express, PostgreSQL, MinIO và Docker Compose; có CRUD, quản lý lô/hạn dùng, bán hàng, upload/download, phân quyền và kiểm chứng dữ liệu bền vững.

BẢN MẪU — CẦN BỔ SUNG TÊN THÀNH VIÊN, ẢNH CHỤP VÀ KẾT QUẢ CHẠY THỰC TẾ TRƯỚC KHI NỘP

1. Giới thiệu bài toán

Cửa hàng mỹ phẩm nhỏ thường quản lý danh mục, nhãn hàng, nhà cung cấp, khách hàng, tồn kho và đơn hàng bằng sổ ghi chép hoặc nhiều bảng tính tách rời. Cách làm này gây khó khăn khi tra cứu mã sản phẩm, kiểm tra lô hàng và hạn sử dụng, tổng hợp doanh thu, cũng như lưu trữ ảnh sản phẩm và chứng từ. Đồ án xây dựng một hệ thống quản lý thống nhất, có thể khởi chạy trên máy cá nhân để trình diễn kiến thức điện toán đám mây.

2. Mục tiêu hệ thống

Mục tiêu chính là xây dựng giao diện có thể sử dụng trên trình duyệt; triển khai REST API; lưu dữ liệu có quan hệ bằng SQL; lưu file qua MinIO; quản lý quyền Administrator/Staff; và chứng minh tính bền vững của dữ liệu bằng Docker volumes. Mục tiêu mở rộng gồm quản lý lô/hạn sử dụng, giao dịch bán hàng có transaction và báo cáo doanh thu.

3. Phạm vi

Phạm vi gồm danh mục, sản phẩm, nhãn hàng, nhà cung cấp, khách hàng, lô hàng, đơn hàng, chi tiết đơn, tệp upload, tài khoản, audit log và báo cáo cơ bản. Không nằm trong phạm vi bắt buộc: thanh toán online, nhiều chi nhánh, tích hợp ERP, Kubernetes, cloud công cộng, high availability hoặc microservices. Đây là ứng dụng học tập, không thay thế phần mềm kế toán hoặc hệ thống bán lẻ thương mại đã được kiểm định.

4. Yêu cầu chức năng

Người dùng đăng nhập và đăng xuất; xem dashboard; tìm kiếm dữ liệu theo mã/tên; tạo, xem, cập nhật và xóa các thực thể được phân quyền; nhập lô hàng và theo dõi hạn dùng; tạo đơn hàng và in hóa đơn; upload, xem và tải tệp; xem báo cáo; Administrator quản lý tài khoản và nhật ký. Các biểu mẫu có nhãn, validation cơ bản và thông báo kết quả. Các thao tác quan trọng được kiểm tra lại ở backend, không chỉ dựa vào giao diện.

5. Yêu cầu phi chức năng

Ứng dụng cần dễ chạy trên Windows với Docker Desktop, có cấu hình thông qua biến môi trường, hạn chế lộ thông tin nhạy cảm, dùng parameterized SQL, hash mật khẩu, xử lý lỗi có thông điệp phù hợp và giao diện co giãn theo kích thước màn hình. Persistence được kiểm chứng bằng việc dừng và chạy lại Compose mà không xóa volume. Cấu hình demo phải dễ tái tạo và tài liệu phải chỉ rõ lệnh khởi động, URL, tài khoản và cách kiểm thử.

6. Use Case

Hai actor chính là Administrator và Staff. Cả hai có thể đăng nhập, xem dashboard, tra cứu dữ liệu, thực hiện nghiệp vụ được cấp quyền, bán hàng và quản lý file theo quyền. Administrator có thêm quyền quản lý tài khoản, dữ liệu danh mục và nhật ký. Sơ đồ nguồn được đặt tại docs/diagrams/use-case.mmd. Khi hoàn thiện báo cáo, nhóm nên render sơ đồ Mermaid thành ảnh PNG để đưa vào tài liệu và slide.

7. Kiến trúc tổng thể

Lựa chọn phương án giao diện tích hợp backend: Node.js/Express phục vụ EJS và REST API trong cùng container. PostgreSQL lưu dữ liệu quan hệ; MinIO lưu nội dung file; Docker Compose khai báo service, environment, network và volumes. Browser chỉ giao tiếp HTTP với ứng dụng và MinIO Console dành cho demo, không kết nối trực tiếp PostgreSQL. Kiến trúc này phù hợp phương án một Dockerfile ứng dụng được nêu trong tài liệu yêu cầu.

Luồng: Browser → Application container (Node.js/Express/EJS) → PostgreSQL và MinIO. Docker Compose tạo network và hai named volumes.

8. Thiết kế database

Cơ sở dữ liệu có các bảng users, categories, brands, suppliers, products, product_batches, customers, promotions, orders, order_items, stock_movements, file_metadata và audit_logs. Khóa chính dùng BIGSERIAL; khóa ngoại thể hiện quan hệ; unique constraint chống trùng mã; check constraint ngăn giá/số lượng âm; index hỗ trợ tìm kiếm và tra cứu lịch sử. Chi tiết được định nghĩa trong database/schema.sql; dữ liệu mẫu nằm trong database/seed.sql.

Bảng nghiệp vụ được liên kết bằng khóa ngoại. Các ràng buộc CHECK và UNIQUE bảo vệ tính hợp lệ; chỉ metadata tệp nằm trong SQL, còn nội dung file nằm trong MinIO.

9. Quản lý sản phẩm

Mỗi sản phẩm có SKU, tên, danh mục, nhãn hàng, nhà cung cấp, mô tả, thành phần, quy cách, giá nhập, giá bán và ngưỡng cảnh báo tồn. Danh sách cho phép tìm theo SKU hoặc tên; trang chi tiết hiển thị thông tin bản ghi. Giá được lưu dưới dạng NUMERIC để tránh sai số nhị phân của số thực. Danh mục, nhãn hàng và nhà cung cấp được tách thành thực thể riêng để giảm trùng lặp dữ liệu.

10. Quản lý lô và hạn sử dụng

Một sản phẩm có thể có nhiều lô nhập. Mỗi lô giữ mã lô, ngày sản xuất, ngày hết hạn, ngày nhận, số lượng nhập, số lượng còn lại và giá nhập. Việc quản lý theo lô giúp xác định sản phẩm sắp hết hạn. Khi bán, hệ thống ưu tiên lô có hạn dùng gần hơn và chỉ sử dụng lô còn hàng, chưa hết hạn. Lịch sử stock_movements ghi nhận nhập, bán và điều chỉnh.

11. Nghiệp vụ đơn hàng

Đơn hàng lưu mã đơn, khách hàng, nhân viên tạo, trạng thái, tạm tính, giảm giá, tổng tiền, phương thức thanh toán và thời điểm hoàn tất. Mỗi chi tiết lưu snapshot SKU/tên, số lượng, giá bán và giá vốn tại thời điểm bán. Việc khóa dòng và cập nhật lô trong transaction giảm nguy cơ bán vượt tồn khi hai thao tác diễn ra đồng thời. Nếu tồn không đủ, lỗi được ném để rollback các thay đổi của đơn.

12. Upload và Object Storage

Tệp được gửi qua multipart/form-data. Backend giới hạn dung lượng, kiểm tra phần mở rộng và MIME type, sinh object key UUID tránh đè tên, rồi lưu nội dung vào bucket MinIO. PostgreSQL chỉ giữ metadata gồm tên gốc, key, MIME, size, bucket, loại/ID thực thể, người upload và thời gian. Nếu ghi metadata thất bại sau khi upload, ứng dụng cố gắng xóa object vừa tạo để tránh file mồ côi.

13. REST API và HTTP status

API cung cấp CRUD cho sản phẩm, danh mục, nhãn hàng, nhà cung cấp và khách hàng; dashboard API trả thống kê; file upload nhận multipart. Các status thường dùng gồm 200, 201, 204, 400, 401, 403, 404 và 500. Tài liệu OpenAPI và Postman Collection được đặt trong thư mục postman. Các request cần session đăng nhập; thao tác quản trị cần role Administrator.

14. Xác thực và phân quyền

Ứng dụng dùng session cookie và bcrypt hash. Tài khoản mẫu được tạo lần đầu dựa trên biến môi trường. Administrator được phép quản lý tài khoản và thao tác quản trị; Staff được dùng các màn hình nghiệp vụ đã cho phép. Mật khẩu không được lưu plaintext. Trong ứng dụng thực tế, cần bổ sung CSRF protection, rate limiting, session store bền vững, chính sách mật khẩu và quy trình đổi mật khẩu.

15. Thiết kế giao diện

Giao diện dùng sidebar, topbar, dashboard cards, bảng dữ liệu, biểu mẫu, trang chi tiết và thông báo. CSS tự thiết kế với màu trung tính, sắc hồng đất và xanh dịu phù hợp chủ đề mỹ phẩm. Trên màn hình nhỏ, sidebar có thể mở/đóng và bảng có vùng cuộn ngang. Các trạng thái rỗng, lỗi và thành công được thể hiện riêng để người dùng dễ hiểu.

16. Dockerfile và Compose

Dockerfile dùng Node.js Alpine, cài dependency production, copy source và khởi chạy ứng dụng. Compose có ba service: application, database và minio. cosmetic-network là bridge network riêng. database-data và minio-data là named volumes. Các thông số nhạy cảm được đọc từ .env, trong khi .env.example chỉ cung cấp tên biến và giá trị mẫu. PostgreSQL được healthcheck trước khi ứng dụng khởi động.

Service application publish 3000; database dùng 5432 nội bộ; MinIO publish 9000 và 9001. Volume database-data và minio-data cần tồn tại sau lệnh docker compose down.

17. Liên hệ VPC/subnet

Docker bridge network tạo vùng giao tiếp nội bộ cho các container. Ứng dụng gọi database bằng DNS service database và gọi MinIO bằng minio, không cần IP tĩnh. Mục tiêu cô lập có điểm tương đồng với VPC/subnet nhưng không thể xem Docker network là VPC hoàn chỉnh: nó không thay thế route table, security group, IAM hay các dịch vụ mạng cloud. Việc phân biệt này cần được nêu rõ trong phần lý thuyết.

Docker bridge network là một cơ chế mạng container có phạm vi giới hạn; không thay thế đầy đủ tính năng VPC/subnet, route table và security group của cloud.

18. Kiểm thử

Cần kiểm thử đăng nhập, phân quyền, CRUD, tìm kiếm, tạo đơn, từ chối bán vượt tồn, upload file hợp lệ/không hợp lệ, xem/tải file, xuất CSV và persistence. Chạy unit test bằng npm test, kiểm tra cú pháp bằng npm run lint:syntax, kiểm tra container bằng docker compose ps và endpoint /health. Ảnh chụp và kết quả thật phải được nhóm bổ sung sau khi chạy trên Docker Desktop; không ghi kết quả giả định thành kết quả đã kiểm chứng.

Bằng chứng cần chèn: ảnh trang dashboard, CRUD, file trong MinIO Console, Swagger/Postman và trạng thái dữ liệu trước/sau khi down/up.

19. Kiểm chứng persistence

Sau khi tạo một bản ghi và upload file, chạy docker compose down, sau đó docker compose up -d. Bản ghi phải còn trong giao diện; file phải còn trong danh sách và có thể tải lại; object vẫn xuất hiện trong MinIO Console. Không dùng docker compose down -v trong bước này vì lệnh đó xóa volumes. Nên ghi lại ảnh chụp trước và sau khi khởi động lại để làm minh chứng báo cáo.

20. Hướng dẫn demo

Trình bày theo thứ tự: bài toán; kiến trúc; khởi chạy Compose; đăng nhập; CRUD tối thiểu ba thực thể; tìm kiếm; tạo lô; bán đơn và in hóa đơn; upload/download; Swagger/Postman; mở MinIO Console; trình bày ERD và Deployment Diagram; cuối cùng chứng minh persistence. Tất cả thành viên nên hiểu phần mình phụ trách và lý do sử dụng volume/network. Thời lượng theo PDF là 10–12 phút demo và 5–7 phút hỏi đáp.

21. Hạn chế

Phiên bản đồ án chưa xử lý đầy đủ đổi trả, hủy đơn, khuyến mãi nâng cao, nhiều chi nhánh, kế toán, backup tự động hoặc triển khai công khai. MinIO đang dùng root credential trong mạng demo; production cần user/policy tối thiểu quyền. Session store mặc định không phù hợp nhiều replica. Đây là các giới hạn cần nêu trung thực khi bảo vệ, đồng thời tránh trình bày dự án như một hệ thống thương mại hoàn chỉnh.

22. Hướng phát triển và kết luận

Có thể mở rộng kiểm thử tích hợp, CSRF, rate limiting, audit đầy đủ hơn, nhập/xuất CSV sản phẩm, hoàn đơn, quản lý chương trình khuyến mãi, báo cáo lợi nhuận, backup định kỳ và triển khai lên VM cloud nếu được yêu cầu. Đồ án minh họa được kiến trúc nhiều tầng, SQL quan hệ, Object Storage, Dockerfile, Compose, biến môi trường và persistence. Trước khi nộp cần xác nhận chạy thành công trên máy nhóm và bổ sung minh chứng thật.

Phụ lục A — Checklist nghiệm thu

Kiểm tra docker compose up -d --build; đăng nhập đúng/sai; quyền Administrator và Staff; CRUD tối thiểu ba thực thể; tìm kiếm; upload/download; tệp hiện trong MinIO; Swagger/Postman; tạo đơn và trừ kho; xử lý lỗi khi tồn không đủ; docker compose down rồi up -d vẫn giữ dữ liệu; source không chứa .env hoặc secret; báo cáo và slide có tên thành viên, phân công và ảnh chụp thực tế.

Tài liệu tham khảo

1. Tài liệu yêu cầu đồ án cuối kỳ học phần Điện toán đám mây do giảng viên cung cấp. 2. Docker Docs — Docker Compose: https://docs.docker.com/compose/. 3. PostgreSQL Documentation: https://www.postgresql.org/docs/. 4. MinIO Documentation: https://min.io/docs/. 5. Express Documentation: https://expressjs.com/.